MONASH University

Searching Smarter, Not Harder

Week 3 — Greedy Best-First Search

Julian Garcia

FIT1061 — Introduction to Artificial Intelligence

COMMONWEALTH OF AUSTRALIA | Copyright Regulations 1969 | WARNING: This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.



© DeepMind / press kit

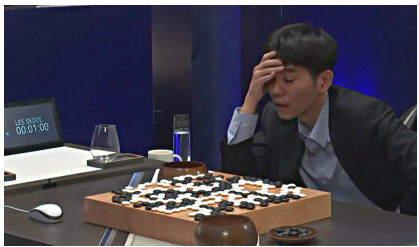
MILESTONE

AlphaGo Move 37

13 March 2016

The move no human would have played.

What happened?

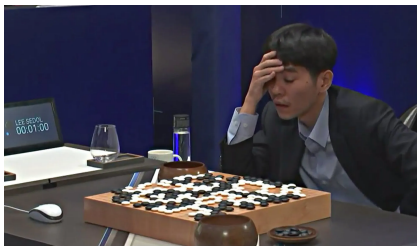


Game 2, AlphaGo vs Lee Sedol (DeepMind press)

Watch: Move 37 — Lee Sedol steps away

DeepMind's **AlphaGo** beat Lee Sedol, the world's top Go player, in a five-game match.

What happened?



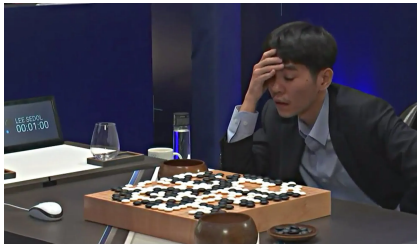
Game 2, AlphaGo vs Lee Sedol (DeepMind press)

▶ Watch: Move 37 — Lee Sedol steps away

DeepMind's **AlphaGo** beat Lee Sedol, the world's top Go player, in a five-game match.

On move 37 of game 2, AlphaGo played a move that **professional commentators called a mistake**.

What happened?



Game 2, AlphaGo vs Lee Sedol (DeepMind press)

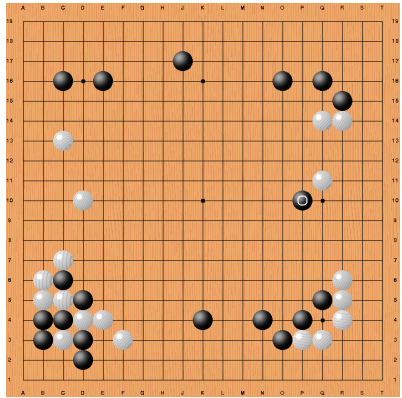
Watch: Move 37 — Lee Sedol steps away

DeepMind's **AlphaGo** beat Lee Sedol, the world's top Go player, in a five-game match.

On move 37 of game 2, AlphaGo played a move that **professional commentators called a mistake**.

It wasn't. AlphaGo won the game.

Move 37

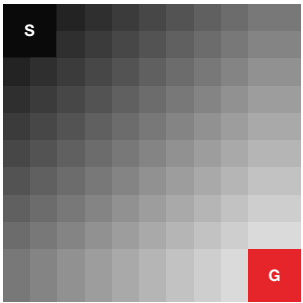


“It’s not a human move. I’ve never seen a human play this move.”

— Fan Hui, professional Go player

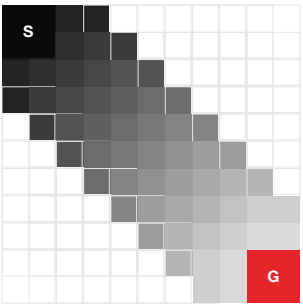
What if the algorithm had a hint?

BFS — blind sweep



~73 cells explored.

With a hint — corridor

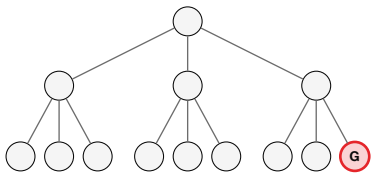


~30 cells explored.

Half the work.

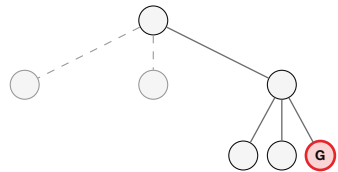
The same idea — as a tree of moves

BFS — expand every branch



13 nodes visited.

With a hint — follow the branch



5 nodes visited.



Today

Board games

Searching with hints

Greedy best-first

Different terrain

Whose intuition?

Board games
Searching with hints
Greedy best-first
Different terrain
Whose intuition?

Board games

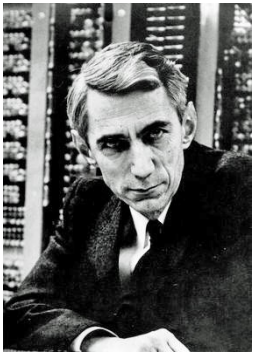
what makes them hard

Why board games?

- **Clear rules, clear goals.** Win, lose, draw. The algorithm's score is unambiguous.
- **Small world, huge depth.** Simple rules, billions of positions. Strategy without messy reality.
- **Safe experimentation.** A million failures cost nothing. Perfect for trial-and-error learning.
- **Built-in benchmark ladder.** Centuries of human play: amateur $\rightarrow$ expert $\rightarrow$ grandmaster $\rightarrow$ superhuman.

Board games are miniature worlds.

1950 — the beginning



Claude Shannon (1916–2001)

Programming a Computer for Playing Chess

Claude Shannon publishes the first formal paper on computer chess. Two strategies:

- **Type A:** brute-force search.
- **Type B: selective search** — focus only on *promising* moves.

Type B requires a way to judge *promising*. A **heuristic**.

Same year: Turing writes a chess program by hand — a 'paper machine' — and plays it against humans, move by move.

1959 — the program that learned



Arthur Samuel (1901–1990)

A Self-Learning Checkers Program

Arthur Samuel's program plays itself thousands of times, adjusting how it scores positions after each game.

It learns to **beat its programmer**.

One of the earliest examples of **machine learning**. The idea: don't tell the machine *how* to play. Let it discover what 'promising' means.

Wave 1 — search wins (1990s)

Year	Program	Game
1992	TD-Gammon (Tesauro)	Backgammon — world-champion via self-play + neural net
1997	**Deep Blue** (IBM)	Chess — defeats Kasparov in match
2007	Chinook (Schaeffer)	Checkers — **solved**: perfect play is a draw

Wave 1 — search wins (1990s)

Year	Program	Game
1992	TD-Gammon (Tesauro)	Backgammon — world-champion via self-play + neural net
1997	**Deep Blue** (IBM)	Chess — defeats Kasparov in match
2007	Chinook (Schaeffer)	Checkers — **solved**: perfect play is a draw

Pattern: massive search + hand-crafted evaluation.

Wave 2 — learning + self-play (2010s)

Year	Program	What changed
2016	**AlphaGo**	Beats Lee Sedol at Go — neural net + tree search
2017	AlphaGo Zero	Beats AlphaGo — learns from *zero* human games
2017	AlphaZero	Same algorithm masters chess, shogi, Go

Wave 2 — learning + self-play (2010s)

Year	Program	What changed
2016	**AlphaGo**	Beats Lee Sedol at Go — neural net + tree search
2017	AlphaGo Zero	Beats AlphaGo — learns from *zero* human games
2017	AlphaZero	Same algorithm masters chess, shogi, Go

Pattern: learned intuition + planning.

Wave 3 — hidden info + language (2020s)

Year	Program	What changed
2017	Libratus (CMU)	Defeats top pros at no-limit Texas hold'em
2019	Pluribus	Beats pros at *multi-player* poker
2022	**CICERO** (Meta)	Human-level Diplomacy: strategy *and* natural-language negotiation

Wave 3 — hidden info + language (2020s)

Year	Program	What changed
2017	Libratus (CMU)	Defeats top pros at no-limit Texas hold'em
2019	Pluribus	Beats pros at *multi-player* poker
2022	**CICERO** (Meta)	Human-level Diplomacy: strategy *and* natural-language negotiation

Pattern: reasoning under uncertainty, with other agents.

The pattern

Era	Games	What the machine had to learn
1950s–60s	Chess, checkers	Search, heuristics, early learning
1990s	Chess, backgammon	Massive search; learning by trial and error
2010s	Go	Neural nets + self-play + tree search
2020s	Poker, Diplomacy	Hidden info, negotiation, multi-agent reasoning

AI moved from **calculation** to **learning** to **self-play** to **interaction**.

The pattern

Era	Games	What the machine had to learn
1950s–60s	Chess, checkers	Search, heuristics, early learning
1990s	Chess, backgammon	Massive search; learning by trial and error
2010s	Go	Neural nets + self-play + tree search
2020s	Poker, Diplomacy	Hidden info, negotiation, multi-agent reasoning

AI moved from **calculation** to **learning** to **self-play** to **interaction**.

But between Wave 1 and Wave 2 was a long pause.

Go resisted for nearly 20 years.

Why?

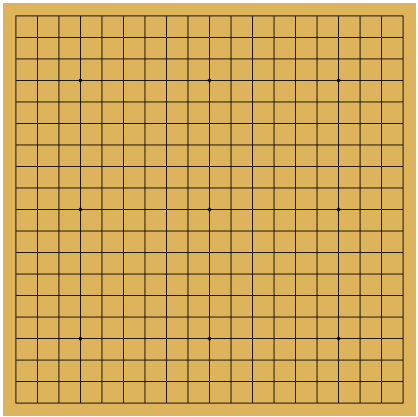
Chess — a quick recap

- Branching factor: ∼**35** legal moves per position.
- Deep Blue searched ∼**6 full moves** ahead.
- **200 million** positions per second. Tens of billions per move.

Brute force, plus a hand-crafted evaluation.

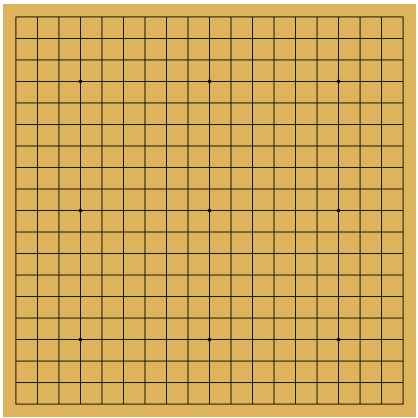
That was enough for chess.

Go — a board, a stone, and a lot of choices



$19 \times 19 = 361$ intersections.

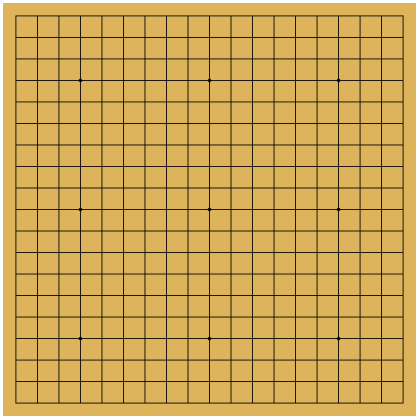
Go — a board, a stone, and a lot of choices



$19 \times 19 = 361$ intersections.

~ 250 legal moves per turn.

Go — a board, a stone, and a lot of choices



$19 \times 19 = 361$ intersections.

~ 250 legal moves per turn.

Average game: ~ 150 **moves**.

The Go explosion

Half-moves	Chess (35 each)	Go (250 each)	Go is like…
1	35	250	a classroom
2	1,225	62,500	a stadium
5	$\sim 5 \times 10^7$	$\sim 10^{12}$	every star in the Milky Way
10	$\sim 3 \times 10^{15}$	$\sim 10^{24}$	atoms in a grain of sand $\times 10^4$
full game	$\sim 10^{120}$	$\sim 10^{170}$	atoms in the observable universe $= 10^{80}$

The Go explosion

Half-moves	Chess (35 each)	Go (250 each)	Go is like…
1	35	250	a classroom
2	1,225	62,500	a stadium
5	$\sim 5 \times 10^7$	$\sim 10^{12}$	every star in the Milky Way
10	$\sim 3 \times 10^{15}$	$\sim 10^{24}$	atoms in a grain of sand $\times 10^4$
full game	$\sim 10^{120}$	$\sim 10^{170}$	atoms in the observable universe $= 10^{80}$

Deep Blue beat chess by searching millions of positions. Go's tree is far too big to search that way.

Bigger *and* harder to judge

Chess engines did not just search. They also **evaluated** positions — ”this side is up a pawn”, ”the king is exposed” — using hand-crafted rules.

Bigger *and* harder to judge

Chess engines did not just search. They also **evaluated** positions — ”this side is up a pawn”, ”the king is exposed” — using hand-crafted rules.

For Go, those hand-crafted rules **did not work**.

- Influence on the board is fuzzy and global.
- A stone’s value depends on patterns 50+ moves later.
- Strong human players talk about *feel*, not formulas.

Bigger *and* harder to judge

Chess engines did not just search. They also **evaluated** positions — ”this side is up a pawn”, ”the king is exposed” — using hand-crafted rules.

For Go, those hand-crafted rules **did not work**.

- Influence on the board is fuzzy and global.
- A stone’s value depends on patterns 50+ moves later.
- Strong human players talk about *feel*, not formulas.

Chess was hard but *search-friendly*.
Go was *both* search-resistant *and* evaluation-resistant.

AlphaGo’s two hints

AlphaGo did not search all $\sim 10^{170}$ positions. It searched smarter — using two **learned hints**.

AlphaGo's two hints

AlphaGo did not search all $\sim 10^{170}$ positions. It searched smarter — using two **learned hints**.

Hint 1 — which moves are worth trying?
Don't consider all $\sim$250 legal moves. Focus on the handful that look promising.

Hint 2 — how good does this position look?
Don't play the game to the end. Estimate the result from *here*.

AlphaGo's two hints

AlphaGo did not search all $\sim 10^{170}$ positions. It searched smarter — using two **learned hints**.

Hint 1 — which moves are worth trying?
Don't consider all $\sim$250 legal moves. Focus on the handful that look promising.

Hint 2 — how good does this position look?
Don't play the game to the end. Estimate the result from *here*.

Both are **heuristics**. AlphaGo's were learned from millions of games.

Go’s game tree has 10^{170} positions.

The observable universe has 10^{80} atoms.

Go's game tree has 10^{170} positions.

The observable universe has 10^{80} atoms.

You cannot search blind.

You need a **hint**.

Board games

Searching with hints

Greedy best-first

Different terrain

Whose intuition?

Searching with hints

What is a heuristic?

Heuristic

A practical **rule of thumb** for making a good decision *without checking every option*.

What is a heuristic?

Heuristic

A practical **rule of thumb** for making a good decision *without checking every option*.

Not perfect. Fast. **Usually good enough.**

The same idea, everywhere

Scenario	What you want to know	The quick rule	When it can mislead
Choosing a restaurant	Is the food good?	It's busy and well-rated	A busy tourist trap
Filtering spam	Is this email junk?	Money words, all caps, odd link	A real email from your bank
Searching a grid	How far to the goal?	Straight-line distance	**A wall blocks the way**

The same idea, everywhere

Scenario	What you want to know	The quick rule	When it can mislead
Choosing a restaurant	Is the food good?	It's busy and well-rated	A busy tourist trap
Filtering spam	Is this email junk?	Money words, all caps, odd link	A real email from your bank
Searching a grid	How far to the goal?	Straight-line distance	**A wall blocks the way**

The rule is cheap to check. **Sometimes it is wrong.**

The heuristic in our search

$h(n)$

A **guess** of remaining cost from node n to the goal.

- Not exact.
- Doesn't have to be precise.
- Just **useful**.

Estimating the distance to the goal

On a grid, every node has a position.

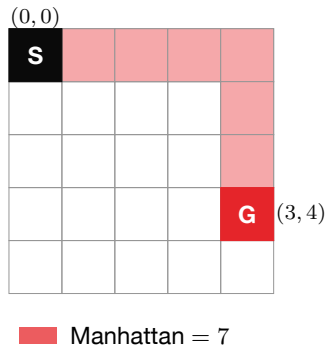
$S = (0, 0)$, $G = (3, 4)$ (row, col)

$\Rightarrow \ \Delta r = 3, \ \Delta c = 4.$

Manhattan distance

$$h_M(n, g) \ = \ |\Delta r| + |\Delta c|$$

$= |3| + |4| =$ **7**. *Count grid steps, like city blocks.*



The estimate comes straight from the two positions.

A second estimate: straight-line

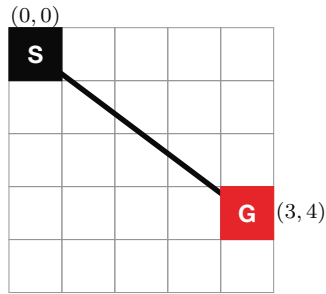
Euclidean distance

$$h_E(n, g) \;=\; \sqrt{(\Delta r)^2 + (\Delta c)^2}$$

$= \sqrt{3^2 + 4^2} = \mathbf{5}$. *Straight-line distance.*

Same two points, a different ruler:

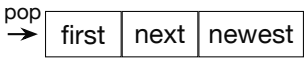
Manhattan $= 7$ (grid steps), Euclidean ≈ 5 (as the crow flies).



In the pseudocode, $h(n, g)$ computes exactly this — from n's position and the goal's.

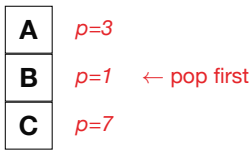
From queue to priority queue

Queue (FIFO)



Whichever item joined *first* comes out first.

Priority queue



Whichever item has the *lowest priority* comes out first.

Same loop. Different scheduler.

Priority queue — interface

PriorityQueue

```
PriorityQueue.push(item, priority)
PriorityQueue.pop() → item
PriorityQueue.is_empty() → true / false
```

Lower priority value = comes out first.

Ties broken by insertion order (FIFO).

Board games

Searching with hints

Greedy best-first

Different terrain

Whose intuition?

Greedy best-first

Poll

A search always steps toward whichever option looks closest to the goal.

Will it always find the shortest path?

- **Always**
- **Usually, but not always**
- **Almost never**

Vote now — scan the code for your campus, *or* type the address.



Melbourne

pollev.com/juliangarcia



Malaysia

pollev.com/onghueyfang767

The idea

- Walk toward the goal.

The idea

- Walk toward the goal.
- Always pick the cell that **looks** closest.

The idea

- Walk toward the goal.
- Always pick the cell that **looks** closest.
- Use the heuristic as the priority.

"Looks closest" = lowest $h(n)$.

One swap from BFS

BFS (last week)

```
function BFS(start, goal):
  frontier ← Queue()
  frontier.enqueue(start)
  visited ← {start}
  came_from ← {start: nil}
  while frontier is not empty:
    current ← frontier.dequeue()
    if current = goal:
      return reconstruct_path(came_from)
    for each neighbour of current:
      if neighbour not in visited:
        visited.add(neighbour)
        came_from[neighbour] ← current
        frontier.enqueue(neighbour)
  return failure
```

Greedy best-first

```
function GREEDY(start, goal, h):
  frontier ← PriorityQueue()
  frontier.push(start, h(start, goal))
  visited ← {start}
  came_from ← {start: nil}
  while frontier is not empty:
    current ← frontier.pop()
    if current = goal:
      return reconstruct_path(came_from)
    for each neighbour of current:
      if neighbour not in visited:
        visited.add(neighbour)
        came_from[neighbour] ← current
        frontier.push(neighbour,
          h(neighbour, goal))
  return failure
```

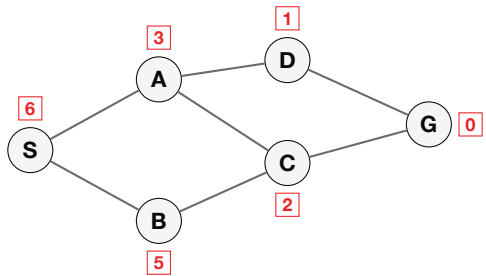
One data-structure swap. The priority is $h(n)$.

Greedy best first search

Greedy best-first

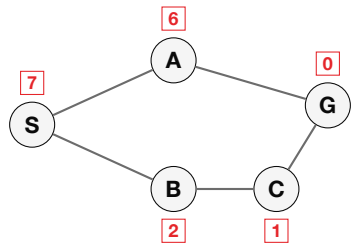
```
function GREEDY(start, goal, h):
  frontier ← PriorityQueue()
  frontier.push(start, h(start, goal))
  visited ← {start}
  came_from ← {start: nil}
  while frontier is not empty:
    current ← frontier.pop()
    if current = goal:
      return reconstruct_path(came_from)
    for each neighbour of current:
      if neighbour not in visited:
        visited.add(neighbour)
        came_from[neighbour] ← current
        frontier.push(neighbour,
          h(neighbour, goal))
  return failure
```

Greedy best-first



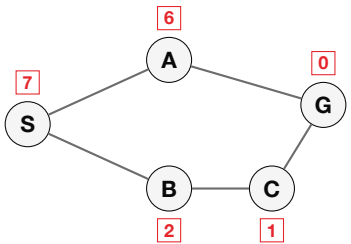
Pop the lowest h. Push neighbours. Stop at **G**.

A worked example



The h-values are a hint. We'll trust them. What happens?

Trace it: greedy from S to G



- Pop the node with the **lowest** h.
- Push its neighbours with their h.
- Stop when we pop **G**.

Poll — vote again

A search always steps toward whichever option looks closest to the goal.

Will it always find the shortest path?

- **Always**
- **Usually, but not always**
- **Almost never**

Vote now — scan the code for your campus, *or* type the address.



Melbourne

pollev.com/juliangarcia

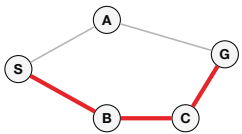


Malaysia

pollev.com/onghueyfang767

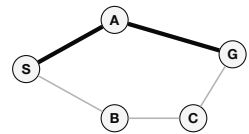
The shortest path was elsewhere

Greedy returned



S → B → C → G (3 edges)

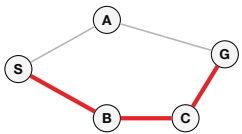
Shortest path



S → A → G (2 edges)

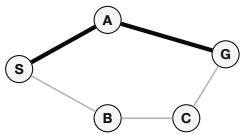
The shortest path was elsewhere

Greedy returned



$S \rightarrow B \rightarrow C \rightarrow G$ (3 edges)

Shortest path



$S \rightarrow A \rightarrow G$ (2 edges)

Greedy is not optimal.

Board games

Searching with hints

Greedy best-first

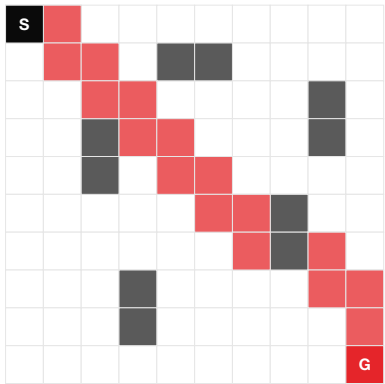
Different terrain

Whose intuition?

Different terrain

same hint, different result

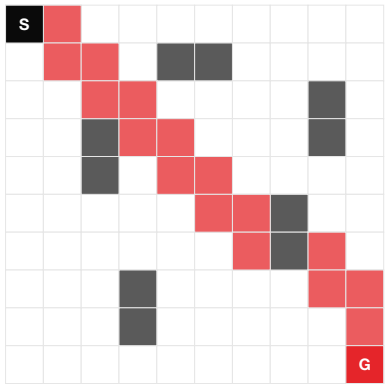
On most grids, greedy wins



	cells
BFS	71
Greedy	**20**

Both find the **18-step shortest path**.

On most grids, greedy wins



	cells
BFS	71
Greedy	**20**

Both find the **18-step shortest path**.

Greedy explored $\sim$**3**$\times$ **fewer cells**.

The trade-off

	BFS	Greedy best-first
Data structure	Queue	Priority queue
Priority	FIFO	$h(n)$
Uses heuristic?	No	Yes
Optimal?	**Always**	Not guaranteed
Speed	Slow	Fast — explores fewer cells

Greedy is **fast and often right**.

When it's wrong, **it's wrong about path length**.

What if we also counted the cost?

- Greedy only looked at $h(n)$ — the guess of what's ahead.

What if we also counted the cost?

- Greedy only looked at $h(n)$ — the guess of what's ahead.
- What if we also counted **what it cost to get here**?

What if we also counted the cost?

- Greedy only looked at $h(n)$ — the guess of what's ahead.
- What if we also counted **what it cost to get here**?

$$f(n) = g(n) + h(n)$$

$g(n)$ = path cost from start. $h(n)$ = heuristic to goal. $f(n)$ = total estimated cost.

A property worth a name — admissibility

Admissible heuristic

A heuristic is **admissible** when it never says the goal is further than it actually is.

A property worth a name — admissibility

Admissible heuristic

A heuristic is **admissible** when it never says the goal is further than it actually is.

Admissible

- Manhattan: 7. Truth ≥ 7. ✓
- Euclidean: ≈ 5. Truth ≥ 5. ✓
- $h(n) = 0$. ✓

Not admissible

- Rectangle area $|\Delta r| \times |\Delta c|$: 12. Truth 7. ×
- Anything that overestimates. ×

A property worth a name — admissibility

Admissible heuristic

A heuristic is **admissible** when it never says the goal is further than it actually is.

Admissible

- Manhattan: 7. Truth ≥ 7. ✓
- Euclidean: ≈ 5. Truth ≥ 5. ✓
- $h(n) = 0$. ✓

Not admissible

- Rectangle area $|\Delta r| \times |\Delta c|$: 12. Truth 7. ×
- Anything that overestimates. ×

Why it matters: an optimistic guess is what guarantees $f = g + h$ finds the *shortest* path.

More about admissibility in the supplementary notes.

See it run: BFS vs greedy vs A*

Same maze. Same hint.

Which one gets **trapped** — and which one **reroutes**?

A* search — the supplementary notes

This is **A***.

It guarantees the shortest path when h is admissible.

Where to find it

Full algorithm, pseudocode, admissibility proof, and maze examples:

`W03_astar_supplement.pdf` — in the unit resources.

Distinction tier. Read it. Then try A* on the lab grid.

Board games

Searching with hints

Greedy best-first

Different terrain

Whose intuition?

Whose intuition?

Whose guess gets built in?

“With the debut of AI in Go games, I’ve realised that I’m not at the top even if I become number one through frantic efforts.”

— Lee Sedol, on retirement (2019)

Whose guess gets built in?

> *"With the debut of AI in Go games, I've realised that I'm not at the top even if I become number one through frantic efforts."*
>
> — Lee Sedol, on retirement (2019)

Discussion

Heuristics are designed by people.

- Manhattan distance assumes only 4 directions. *Whose world is that?*
- Euclidean distance assumes empty space. *Whose world?*
- AlphaGo learned what *looks* promising from a database of human games. *Whose* games? What style got reinforced?

In the lab this week

Pass tier. **Aiming for a Pass**

1. Implement Manhattan and Euclidean distance.
2. Build a priority queue.
3. Implement greedy best-first search — one swap from BFS.
4. Hand-trace greedy on a small graph.
5. Compare greedy vs BFS on a 10×10 grid: which is shorter? Which expands fewer cells?

Distinction tier. **Aiming for a Distinction** — all of the above, plus: read the A* supplementary notes, implement A*, design your own admissible heuristic, and find a grid where A* and BFS find the same path but A* expands far fewer cells.

The hint can point the wrong way.

Better hints, better search.

The hint can point the wrong way.

Better hints, better search.

Next week: what if we abandon the frontier entirely?

Hill climbing.